

Object-Oriented Programming

Lecture 5: OOP II

Prof. Jaehyung Seo

(seojae777@konkuk.ac.kr)

Language Intelligence & Representation Lab.

Overview

Topics

Abstract Base Classes (ABCs) — Deep Dive

- Beyond L4's Distance ABC: `collections.abc`, protocols, and type hints

Operator Overloading

- Mapping Python operators (+, /, []) to special methods

Extending Built-ins — Advanced Patterns

- Building on L4's subclassing: `__getitem__`, `__contains__`, and custom containers

Metaclasses

- Modifying class creation at the type level

Creating an Abstract Base Class (ABC)

*In Lecture 4, we used `abc.ABC` to define the *Distance* base class for KNN.*

Formal Definition

- An Abstract Base Class (ABC) acts as a template or interface. It dictates **what methods** a subclass must implement, but does not provide the full implementation itself.

Beyond Distance: A New Example

- A media player with third-party plugins. An ABC documents the exact API (like a `play()` method) that all plugins must provide.

The @abstractmethod Decorator

Code Snippet:

```
import abc
```

```
class MediaLoader(abc.ABC):  
    @abc.abstractmethod  
    def play(self) -> None:  
        pass  
    @property  
    @abc.abstractmethod  
    def ext(self) -> str:  
        pass
```

```
class Wav(MediaLoader):  
    pass
```

```
# w = Wav() # TypeError: Can't instantiate abstract class Wav
```

The collections.abc Module

ABCs in collections.abc

Key Takeaway

- Python provides pre-built ABCs for all standard data structures in the collections.abc module.

If you are building a custom list, dictionary, or set, you should inherit from these ABCs (like Sequence, Mapping, or Set) to guarantee seamless integration with Python's native functions.

Validating Protocols with collections.abc

Code Snippet:

```
import collections.abc

# A standard dictionary is a Mapping
my_dict = {"a": 1, "b": 2}
print(isinstance(my_dict, collections.abc.Mapping))
# True

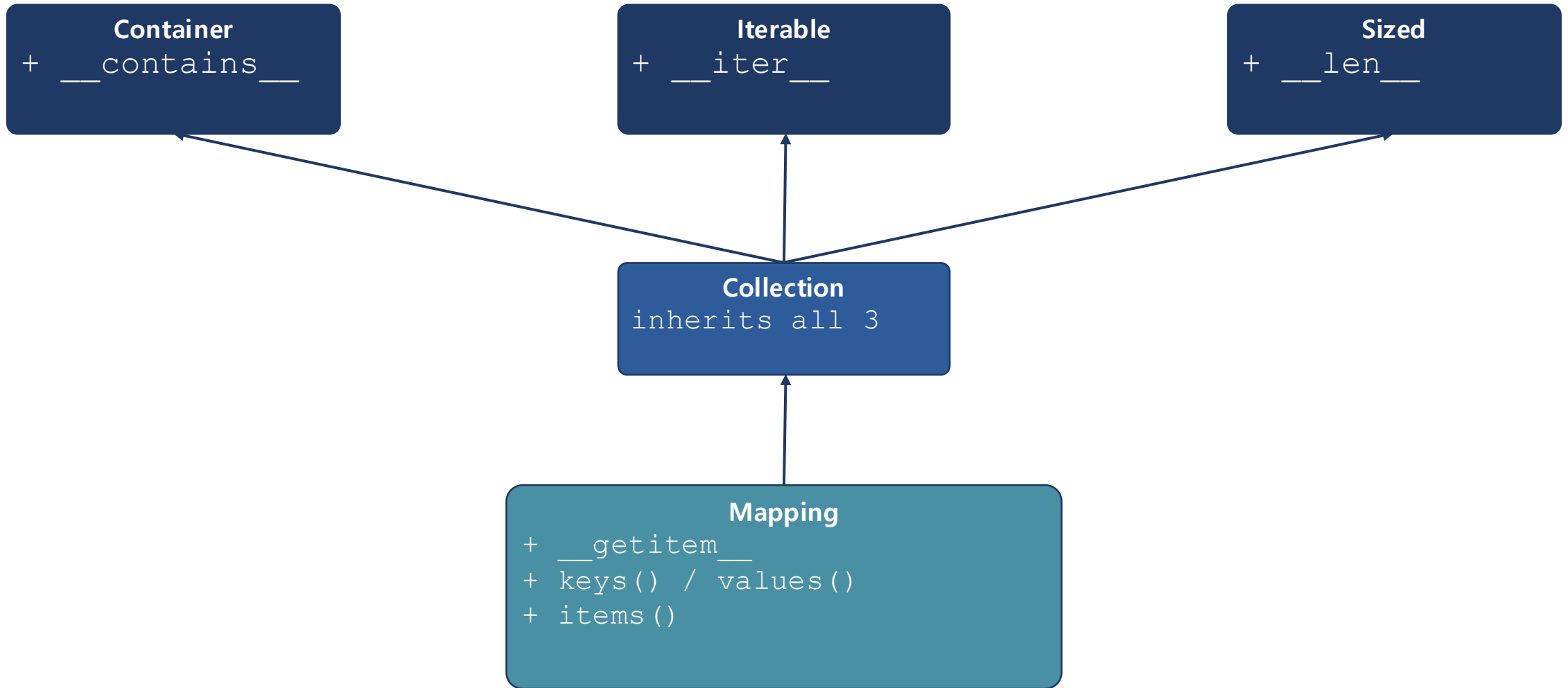
# Use ABCs as base classes for our own types
class MyCustomMap(collections.abc.Mapping):
    # Must implement __getitem__, __iter__, __len__
    pass
```

ABCs as Type Hints

ABCs are closely tied to **generic type hints**.

- An ABC like `Iterable[str]` acts as a formal contract, telling mypy exactly what behaviors an object must possess.
- There is a world of difference between `list[Any]` and `list[int]`.
- Using `Iterable` emphasizes we only need the iteration aspect, regardless of whether it is a list, a file, or a custom object.

The Mapping Hierarchy



Arrows point to parent ABC (inheritance). Interface Segregation Principle in action.

A Custom Lookup Class

Code Snippet:

```
from collections import abc
from typing import Any, Iterable
```

```
# Type Alias for clarity
BaseMapping = abc.Mapping[str, Any]
```

```
class Lookup(BaseMapping):
    def __init__(self, source: Iterable[tuple[str, Any]]):
        sorted_pairs = sorted(source, key=lambda x: x[0])
        self.key_list = [k for k, v in sorted_pairs]
        self.value_list = [v for k, v in sorted_pairs]
```

Fulfilling the ABC Contract

Implementing a Mapping

Steps to make `Lookup` a valid Mapping:

- Implement `__len__` (from Sized ABC)
- Implement `__iter__` (from Iterable ABC)
- Implement `__getitem__` (from Mapping ABC)
- `__contains__` is provided for free by Mapping using `__getitem__`, but overriding it can improve performance.

Implementing Abstract Methods

Code Snippet:

```
import bisect

class Lookup(BaseMapping):
    # ... (__init__ omitted) ...

    def __len__(self) -> int:
        return len(self.key_list)

    def __iter__(self):
        return iter(self.key_list)

    def __getitem__(self, key: str) -> Any:
        index = bisect.bisect_left(self.key_list, key)
        if index < len(self.key_list) and \
            self.key_list[index] == key:
            return self.value_list[index]
        raise KeyError(key)
```

Defining Custom ABCs

Creating Your Own ABC

Definition

- Creating your own ABC leverages the `abc` module. The `abc.ABC` class definition is the perfect starting point.

Example

- Creating an abstract `Die` class that defines a `roll()` method. Specific dice like D4 or D6 must implement the specific random selection logic.

The Die Hierarchy

Code Snippet:

```
import abc
import random

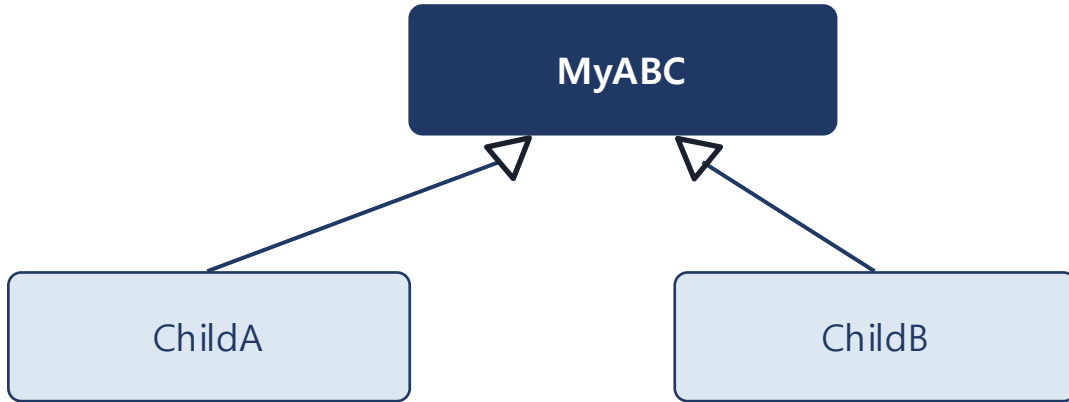
class Die(abc.ABC):
    def __init__(self) -> None:
        self.face: int
        self.roll()

    @abc.abstractmethod
    def roll(self) -> None:
        pass

class D6(Die):
    def roll(self) -> None:
        self.face = random.randint(1, 6)
```

Nominal vs. Structural Subtyping — Visual

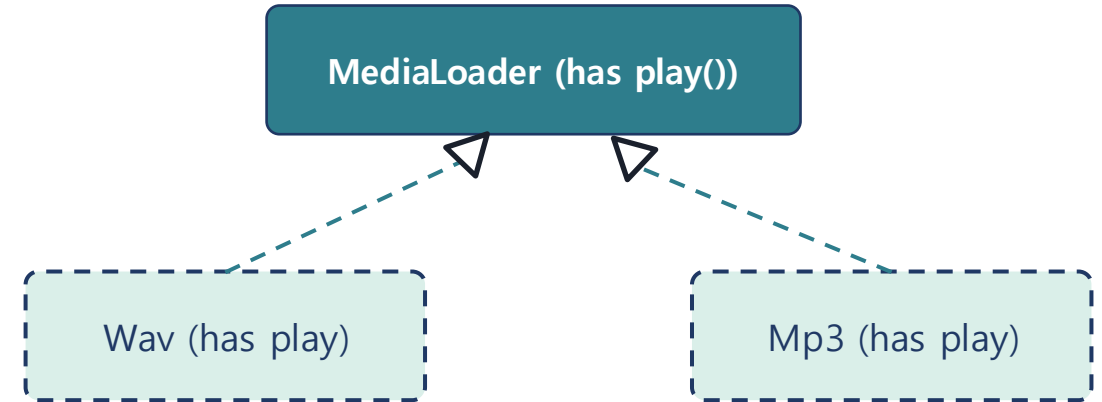
NOMINAL (by inheritance)



explicit `'class ChildA(MyABC):'` → `isinstance` ✓

Python checks: is `MyABC` in `ChildA.__mro__` ?

STRUCTURAL (by methods)



No `'class Wav(MediaLoader)'` needed → `isinstance` ✓

`__subclasshook__` checks: does `'play'` exist in `C.__mro__` ?

Solid arrow = real inheritance (nominal). Dashed arrow = duck-typed match via `__subclasshook__` (structural).